SQL Server Performance Tuning: A Practical SOP for Bottleneck Analysis (2016–2025)

> **Scope:** SQL Server 2016 through SQL Server 2025 (on-premises and Azure SQL Managed Instance) **Audience:** DBAs, Platform Engineers, and Backend Developers responsible for production SQL Server environments **Purpose:** Step-by-step Standard Operating Procedure (SOP) for diagnosing and resolving performance bottlenecks systematically — not reactively

---

Table of Contents

---

1. The Core Principle: Structure Before Action

Production performance problems arrive with pressure. Users are waiting. Jobs are queued. Management wants answers in five minutes. The fastest-looking response — jumping

straight to a query plan, killing a blocking session, or adding an index — is almost never the right one.

The visible symptom is a clue, not a diagnosis.

A CPU spike can mean parameter sniffing, a plan regression after a stats update, increased concurrency, or a missing index. A dominant wait type means nothing without knowing *which workload* produced it, *which time window*, and *compared to what baseline*. A blocking chain tells you *who* is blocked but rarely *why* the transaction held locks longer than expected.

This SOP enforces a four-phase gate before any change touches production:

```
Problem Definition → Triage → Evidence → Change (small, documented, validated)
```

Every minute spent in the first three phases saves ten minutes of undoing the wrong fix.

---

2. Phase 1 — Problem Definition

Before opening SSMS, answer these five questions in writing (Slack, incident ticket, or a notepad):

Question	Why it matters
What is affected?	Single query / procedure / app feature / whole instance?
When did it start?	Timestamp, not "sometime today." Match to deployments, agent jobs, index rebuilds, statistics updates.
Is the behavior constant or periodic?	Constant = structural problem. Periodic = load-related, schedule-related, or plan-cache pollution.
What changed recently?	Deployment, schema change, data volume spike, server patching, compatibility level change, hardware migration.
What does "slow" mean numerically?	"The order search proc used to finish in 800ms; it is now taking 14 seconds during business hours."

Example of a useful problem statement:

> *"The stored procedure `usp_GetOrdersByCustomer` takes 14 seconds during 09:00–11:00 business hours on the ORDERS database on SQL01. It ran under 1 second yesterday. A*

> *deployment was pushed at 21:00 last night. The issue started at 09:05 this morning. Not reproduced on DEV."*

Example of a useless problem statement:

> *"SQL is slow."*

The first statement gives you: workload identity, time window, baseline, comparison point, and change context. Start the stopwatch here, not at the query plan.

---

3. Phase 2 — Triage and Scope Reduction

Run these queries in order. Each narrows scope before you go deeper.

Step 1 — Is the problem instance-wide or database-specific?

```
-- Active waits right now, grouped by session
SELECT
    r.session_id,
    r.status,
    r.wait_type,
    r.wait_time / 1000.0          AS wait_sec,
    r.cpu_time / 1000.0           AS cpu_sec,
    r.logical_reads,
    r.blocking_session_id,
    DB_NAME(r.database_id)        AS database_name,
    SUBSTRING(st.text, (r.statement_start_offset/2)+1,
        ((CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
          ELSE r.statement_end_offset END - r.statement_start_offset)/2)+1)
                                  AS current_statement
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE r.session_id > 50
  AND r.status <> 'background'
ORDER BY r.wait_time DESC;
```

Interpret:

- If waits are spread across multiple databases → likely instance-level resource pressure (CPU, memory, storage, max worker threads).
- If waits cluster to one database → investigate that database specifically.
- If `blocking_session_id` is non-zero → jump to Scenario 4 (Blocking).

Step 2 — Top cumulative wait types at the instance level

```
-- Baseline: run once, wait 60 seconds, run again and compare delta
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    wait_time_ms / 1000.0                                   AS total_wait_sec,
    (wait_time_ms - signal_wait_time_ms) / 1000.0           AS resource_wait_sec,
    signal_wait_time_ms / 1000.0                            AS signal_wait_sec,
    ROUND(100.0 * wait_time_ms / SUM(wait_time_ms) OVER(), 2) AS pct_total
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
    -- Benign waits to exclude
    'SLEEP_TASK','BROKER_TO_FLUSH','BROKER_TASK_STOP','CLR_AUTO_EVENT',
    'DISPATCHER_QUEUE_SEMAPHORE','FT_IFTS_SCHEDULER_IDLE_WAIT',
    'HADR_FILESTREAM_IOMGR_IOCOMPLETION','HADR_WORK_QUEUE',
    'ONDEMAND_TASK_QUEUE','REQUEST_FOR_DEADLOCK_SEARCH',
    'RESOURCE_QUEUE','SERVER_IDLE_CHECK','SLEEP_DBSTARTUP',
    'SLEEP_DCOMSTARTUP','SLEEP_MASTERDBREADY','SLEEP_MASTERMDREADY',
    'SLEEP_MASTERUPGRADED','SLEEP_MSDBSTARTUP','SLEEP_SYSTEMTASK',
    'SLEEP_TEMPDBSTARTUP','SNI_HTTP_ACCEPT','SP_SERVER_DIAGNOSTICS_SLEEP',
    'SQLTRACE_BUFFER_FLUSH','SQLTRACE_INCREMENTAL_FLUSH_SLEEP',
    'WAIT_XTP_OFFLINE_CKPT_NEW_LOG','WAITFOR','XE_DISPATCHER_WAIT',
    'XE_TIMER_EVENT','BROKER_EVENTHANDLER','CHECKPOINT_QUEUE',
    'DBMIRROR_EVENTS_QUEUE','SQLTRACE_WAIT_ENTRIES','WAIT_XTP_CKPT_CLOSE'
)
ORDER BY wait_time_ms DESC;
```

Route to the right scenario:

Top Wait Type	Jump To
`SOS_SCHEDULER_YIELD`, `CXPACKET`, `CXCONSUMER`	Scenario 1 — CPU / Parallelism
`PAGEIOLATCH_SH`, `PAGEIOLATCH_EX`, `WRITELOG`	Scenario 2 — I/O Pressure
`RESOURCE_SEMAPHORE`, `RESOURCE_SEMAPHORE_QUERY_COMPILE`	Scenario 3 — Memory Pressure
`LCK_M_*`, `SLEEP_*`	Scenario 4 — Blocking / Locking
`ASYNC_NETWORK_IO`	Scenario 5 — Application-side slowness
`THREADPOOL`	Scenario 6 — Worker Thread Exhaustion

`HADR_SYNC_COMMIT`, `HADR_DATABASE_FLOW_CONTROL`	Scenario 7 — AG / Log shipping latency

Step 3 — Check signal waits (CPU pressure indicator)

```
-- Signal wait > 25% strongly suggests CPU pressure
SELECT
    CAST(100.0 * SUM(signal_wait_time_ms) / SUM(wait_time_ms) AS DECIMAL(5,2))
        AS signal_wait_pct
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN ('SLEEP_TASK','WAITFOR','XE_TIMER_EVENT');
```

> **Rule of thumb:** Signal wait percentage above 15–25% indicates CPU is a primary constraint, even if CPU counters don't look extreme.

---

4. Phase 3 — Evidence Collection by Wait Type

Scenario 1 — CPU Pressure / Excessive Parallelism

Real-world case: *An ETL stored procedure that ran fine overnight is running during business hours and consuming 8 of 8 CPU cores. Other sessions are waiting on* `SOS_SCHEDULER_YIELD`.

Diagnosis queries:

```
-- Top CPU-consuming queries right now
SELECT TOP 10
    qs.total_worker_time / qs.execution_count AS avg_cpu_us,
    qs.execution_count,
    qs.total_worker_time,
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
          ELSE qs.statement_end_offset END - qs.statement_start_offset)/2)+1)
                                            AS query_text,
    qs.plan_handle
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY qs.total_worker_time DESC;

-- Check MAXDOP and cost threshold settings
SELECT name, value_in_use
FROM sys.configurations
```

```
WHERE name IN ('max degree of parallelism','cost threshold for parallelism');
```

Common causes and fixes:

Cause	Fix	Version Notes
MAXDOP = 0 on high-core servers	Set MAXDOP per Microsoft formula (typically 8 or half socket count)	All versions; use `ALTER DATABASE SCOPED CONFIGURATION` (2016+)
Cost threshold for parallelism too low (default 5)	Raise to 25–50	All versions
Missing index causes full scan in parallel	Add covering index	All versions
Parameter sniffing → wrong plan using parallelism	Query Store plan forcing or `OPTIMIZE FOR` hint	2016+; use PSP optimization in 2022+

```
-- Database-scoped MAXDOP (SQL Server 2016+)
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 8;

-- Raise cost threshold (instance-level)
EXEC sp_configure 'cost threshold for parallelism', 50;
RECONFIGURE;
```

Scenario 2 — I/O Pressure

Real-world case: *Reports running against a 500GB database take 3x longer after a data load.* `PAGEIOLATCH_SH` *is the top wait type. Storage team says disk latency is normal.*

Diagnosis queries:

```
-- Per-file I/O latency (should be < 10-15ms for data, < 1ms for log)
SELECT
    DB_NAME(vfs.database_id)    AS database_name,
    mf.physical_name,
    mf.type_desc,
    vfs.io_stall_read_ms / NULLIF(vfs.num_of_reads, 0)    AS avg_read_ms,
    vfs.io_stall_write_ms / NULLIF(vfs.num_of_writes, 0)  AS avg_write_ms,
    vfs.num_of_reads,
```

```
    vfs.num_of_writes,
    vfs.io_stall
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf
    ON vfs.database_id = mf.database_id
   AND vfs.file_id = mf.file_id
ORDER BY vfs.io_stall DESC;

-- Find queries generating the most logical reads (buffer pool pressure)
SELECT TOP 10
    total_logical_reads / execution_count AS avg_logical_reads,
    execution_count,
    total_logical_reads,
    SUBSTRING(st.text, (qs.statement_start_offset/2)+1,
        ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
          ELSE qs.statement_end_offset END - qs.statement_start_offset)/2)+1) AS
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY avg_logical_reads DESC;
```

Microsoft guidance: PAGEIOLATCH waits consistently above 10–15ms per I/O request indicate an I/O performance problem requiring further investigation. Values above 1 second have been observed in severely degraded environments.

Common causes and fixes:

Cause	Fix
Table or index scan due to missing index	Add covering index; verify with execution plan
Insufficient buffer pool (SQL caching cold)	Check `sys.dm_os_buffer_descriptors`; increase max server memory
TempDB contention (PFS/GAM/SGAM pages)	Add TempDB data files = number of logical CPUs (up to 8)
Log drive shared with data drive	Separate transaction log to dedicated spindle/SSD
Backup I/O competing with workload	Schedule backups off-peak; use `MAXTRANSFERSIZE` and `BUFFERCOUNT`

```
-- TempDB file count check
SELECT COUNT(*) AS tempdb_data_files
FROM sys.master_files
```

```
WHERE database_id = 2 AND type = 0;

-- If < CPU count (max 8), add files in tempdb
USE tempdb;
ALTER DATABASE tempdb ADD FILE (
    NAME = 'tempdev2',
    FILENAME = 'D:\TempDB\tempdev2.ndf',
    SIZE = 4096MB, FILEGROWTH = 512MB
);
```

Scenario 3 — Memory Pressure / Memory Grants

Real-world case: *Nightly batch queries spill to TempDB.* `RESOURCE_SEMAPHORE` *waits are high. Sort and hash operations in execution plans show warning icons.*

Diagnosis queries:

```
-- Check memory grant usage and spills
SELECT
    r.session_id,
    mg.granted_memory_kb / 1024.0   AS granted_mb,
    mg.used_memory_kb / 1024.0      AS used_mb,
    mg.ideal_memory_kb / 1024.0     AS ideal_mb,
    r.wait_type,
    SUBSTRING(st.text, 1, 200)      AS query_text
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_query_memory_grants mg
    ON r.session_id = mg.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE mg.granted_memory_kb > 0
ORDER BY mg.granted_memory_kb DESC;

-- Check for TempDB spills in Query Store (2016+)
SELECT TOP 10
    qt.query_sql_text,
    rs.avg_num_physical_io_reads,
    rs.avg_spills,                  -- SQL Server 2017+
    rs.count_executions,
    p.plan_id
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q   ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p    ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
ORDER BY rs.avg_spills DESC;        -- Focus on highest-spill queries
```

```
-- PLE (Page Life Expectancy) -- healthy > 300 seconds (higher on large servers)
SELECT object_name, counter_name, cntr_value
FROM sys.dm_os_performance_counters
WHERE counter_name = 'Page life expectancy'
  AND object_name LIKE '%Buffer Manager%';
```

Common causes and fixes:

Cause	Fix
Max server memory set too high (OS starved)	Leave ~10% or 4–10 GB for OS; use Resource Governor for workloads
Cardinality estimate wrong → bad memory grant	Update statistics; in 2022+ CE Feedback fixes this automatically at compat level 160
Sort/hash spills to TempDB	Add indexes to avoid sorts; use Memory Grant Feedback (2019+)
Lock pages in memory not set	Enable on high-memory servers to prevent OS from paging SQL buffer pool

```
-- Set max server memory (leave headroom for OS)
EXEC sp_configure 'max server memory (MB)', 28672;  -- example: 28 GB on 32 GB se
RECONFIGURE;

-- Check cardinality estimator version in use
SELECT compatibility_level FROM sys.databases WHERE name = DB_NAME();
-- Level 130 = SQL 2016 CE; Level 140 = 2017; Level 150 = 2019; Level 160 = 2022
```

Scenario 4 — Blocking and Deadlocks

Real-world case: *Customer-facing order entry screen hangs for 30–60 seconds at peak. Users see "spinning wheel." Support escalates.* `LCK_M_S` *is the top wait type.*

Diagnosis queries:

```
-- Find the full blocking chain
WITH BlockingChain AS (
    SELECT
        s.session_id,
        r.blocking_session_id,
        s.login_name,
```

```
        s.host_name,
        r.wait_type,
        r.wait_time / 1000.0    AS wait_sec,
        r.status,
        SUBSTRING(st.text, (r.statement_start_offset/2)+1,
            ((CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
              ELSE r.statement_end_offset END - r.statement_start_offset)/2)+1) A
        CAST(r.open_transaction_count AS VARCHAR) AS open_txn
    FROM sys.dm_exec_sessions s
    LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
    OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) st
    WHERE s.session_id > 50
)
SELECT * FROM BlockingChain WHERE blocking_session_id > 0 OR session_id IN (
    SELECT blocking_session_id FROM BlockingChain WHERE blocking_session_id > 0
)
ORDER BY blocking_session_id, session_id;

-- Find long-running open transactions (the head blockers)
SELECT
    s.session_id,
    s.login_name,
    s.host_name,
    s.program_name,
    t.transaction_begin_time,
    DATEDIFF(SECOND, t.transaction_begin_time, GETDATE()) AS txn_age_sec,
    SUBSTRING(st.text, 1, 500) AS last_sql
FROM sys.dm_exec_sessions s
JOIN sys.dm_tran_session_transactions tst ON s.session_id = tst.session_id
JOIN sys.dm_tran_active_transactions t    ON tst.transaction_id = t.transaction_i
OUTER APPLY sys.dm_exec_sql_text(s.most_recent_sql_handle) st
WHERE t.transaction_type = 1  -- Read-Write
ORDER BY txn_age_sec DESC;
```

Important — don't just kill the blocker. First understand *why* the transaction is open long. Common patterns:

Pattern	Root Cause	Fix
Head blocker is an idle session with open transaction	Application not committing after user action	Fix application transaction handling; set `READ_COMMITTED_SNAPSHOT`
INSERT/UPDATE/DELETE blocking SELECT	Missing `READ_COMMITTED_SNAPSHOT` isolation	`ALTER DATABASE db SET READ_COMMITTED_SNAPSHOT ON`

		(SQL 2016+ supported)
Long-running report blocking OLTP	Report runs in `SERIALIZABLE` implicitly	Use `WITH (NOLOCK)` on reporting queries, or isolate to a read replica (AG)
Deadlock loop	Circular lock dependency	Enable deadlock trace flag or use XE `xml_deadlock_report`; review index access order

```
-- Enable RCSI (non-blocking readers, recommended for OLTP)
-- Note: Requires brief schema lock on database; plan during maintenance window
ALTER DATABASE [YourDatabase] SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK AFTER

-- Capture deadlocks with Extended Events (2012+, preferred over trace flags)
CREATE EVENT SESSION [DeadlockCapture] ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.ring_buffer(SET max_memory = 51200)
WITH (STARTUP_STATE = ON);
ALTER EVENT SESSION [DeadlockCapture] ON SERVER STATE = START;
```

Scenario 5 — Parameter Sniffing

Real-world case: *The stored procedure `usp_SearchProducts` returns instantly for most users but takes 45 seconds for specific customers. Clearing the plan cache fixes it temporarily but the problem returns within hours.*

This is one of the most common and misdiagnosed problems in SQL Server environments.

How it happens: SQL Server compiles and caches a plan based on the first set of parameters it sees. If those parameters are not representative of the full data distribution, some subsequent calls get a plan optimized for the wrong shape of data.

```
-- Find queries with high variance between min and max execution time
-- (a red flag for parameter sniffing)
SELECT TOP 20
    qs.max_elapsed_time / 1000.0   AS max_elapsed_ms,
    qs.min_elapsed_time / 1000.0   AS min_elapsed_ms,
    qs.total_elapsed_time / qs.execution_count / 1000.0 AS avg_elapsed_ms,
    qs.execution_count,
    qs.plan_generation_num,
    SUBSTRING(st.text, 1, 300)     AS query_text,
    qs.plan_handle
```

```
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE qs.max_elapsed_time > 5 * qs.min_elapsed_time   -- Max is 5x min
  AND qs.execution_count > 10
ORDER BY qs.max_elapsed_time DESC;

-- Get the actual cached plan
SELECT * FROM sys.dm_exec_query_plan(<plan_handle from above>);
```

Fix options by SQL Server version:

Fix	When to use	Version
`OPTION (OPTIMIZE FOR UNKNOWN)`	When no single parameter set is representative	2016+
`OPTION (RECOMPILE)`	Low-frequency stored procedures; when plan reuse isn't beneficial	All
Query Store plan forcing	Pin a known-good plan without code changes	2016+
`WITH RECOMPILE` on stored procedure	Last resort; loses all plan caching benefit	All
Parameter Sensitive Plan (PSP) Optimization	Data skew causing plan conflicts	**SQL Server 2022+ (compat 160)**
Query Store Hints	Apply hints without touching application code	2022+

```
-- PSP Optimization: SQL Server 2022+ handles this automatically at compat level
-- Verify it's active:
SELECT name, value
FROM sys.database_scoped_configurations
WHERE name = 'PARAMETER_SENSITIVE_PLAN_OPTIMIZATION';

-- Query Store Hint (SQL Server 2022+) - apply hint without changing code
EXEC sys.sp_query_store_set_hints
    @query_id = 42,
    @query_hints = N'OPTION(OPTIMIZE FOR UNKNOWN)';
```

Scenario 6 — Expensive Execution Plans / Missing Indexes

Real-world case: *After a monthly data load (table grew from 50M to 200M rows), a batch process that ran in 2 minutes now runs for 40 minutes. No code changed.*

```
-- Missing index recommendations from the query optimizer
SELECT TOP 20
    ROUND(migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks + mi
                                        AS estimated_improvement,
    mid.statement               AS [table],
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.user_seeks,
    migs.user_scans,
    migs.avg_user_impact
FROM sys.dm_db_missing_index_group_stats migs
JOIN sys.dm_db_missing_index_groups mig   ON migs.group_handle = mig.index_group_
JOIN sys.dm_db_missing_index_details mid  ON mig.index_handle = mid.index_handle
ORDER BY estimated_improvement DESC;

-- Unused indexes (candidates for removal - check carefully)
SELECT
    OBJECT_NAME(i.object_id)   AS table_name,
    i.name                     AS index_name,
    i.type_desc,
    ius.user_seeks,
    ius.user_scans,
    ius.user_lookups,
    ius.user_updates            -- High updates + low reads = maintenance overhea
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats ius
    ON i.object_id = ius.object_id AND i.index_id = ius.index_id
    AND ius.database_id = DB_ID()
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
  AND (ius.user_seeks IS NULL OR (ius.user_seeks + ius.user_scans + ius.user_look
  AND i.is_primary_key = 0
  AND i.is_unique_constraint = 0
ORDER BY ius.user_updates DESC;
```

Key rules for index decisions:

- Never add an index from the missing index DMV without reviewing the full workload context. DMVs reset at instance restart and don't account for write overhead.
- High `user_updates` with zero seeks = expensive dead weight. Remove after validating

no app logic depends on it.

- A Key Lookup in an execution plan means a nonclustered index exists but lacks included columns. Add the looked-up columns to the `INCLUDE` clause.
- After data volume doubles, rerun `UPDATE STATISTICS` before diagnosing plan issues.

```
-- Force statistics update with full scan after major data load
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;

-- Or update all tables in a database
EXEC sp_updatestats;

-- Check last stats update date
SELECT
    OBJECT_NAME(s.object_id) AS table_name,
    s.name                   AS stat_name,
    sp.last_updated,
    sp.rows,
    sp.rows_sampled,
    sp.modification_counter
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE OBJECT_NAME(s.object_id) = 'Orders'
ORDER BY sp.last_updated DESC;
```

Scenario 7 — TempDB Contention

Real-world case: *With 64 CPUs and a heavily concurrent OLTP workload, sessions are waiting on `PAGELATCH_UP` waits on pages 1:1:1, 1:1:2, and 1:1:3 — the PFS, GAM, and SGAM allocation pages in TempDB.*

```
-- Check for PFS/GAM/SGAM contention in TempDB
SELECT
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    wait_resource
FROM sys.dm_os_wait_stats
WHERE wait_type LIKE 'PAGELATCH%'
ORDER BY wait_time_ms DESC;

-- Confirm with live latch waits
SELECT
```

```
    t.session_id,
    wt.wait_type,
    wt.wait_duration_ms,
    wt.resource_description
FROM sys.dm_os_waiting_tasks wt
JOIN sys.dm_exec_sessions t ON wt.session_id = t.session_id
WHERE wt.wait_type LIKE 'PAGELATCH%'
  AND wt.resource_description LIKE '2:%';  -- database_id 2 = tempdb
```

Fix: Add TempDB data files. Microsoft recommends one file per logical CPU core, up to 8. SQL Server 2016+ automatically pre-sizes all files equally.

```
-- In SQL Server 2016+: enable trace flag 1117 behavior is default for TempDB
-- Verify TempDB file count and sizes
SELECT name, size * 8 / 1024 AS size_mb, physical_name
FROM sys.master_files
WHERE database_id = 2 AND type = 0;
```

SQL Server 2019+: If you see `PAGELATCH_UP` on TempDB with modern versions and properly sized files, check whether `tempdb_metadata_memory_optimized` is enabled. It can dramatically reduce this contention.

```
-- SQL Server 2019+: Memory-optimized TempDB metadata
ALTER SERVER CONFIGURATION SET TEMPDB METADATA MEMORY_OPTIMIZED = ON;
-- Requires service restart
```

---

5. Phase 4 — Root Cause Scenarios with T-SQL Playbooks

Scenario 8 — Query Regression After SQL Server Upgrade / Compatibility Level Change

Real-world case: *After upgrading from SQL Server 2014 (compat level 120) to SQL Server 2019 (compat level 150), 15 queries regressed. The team is considering rolling back.*

Root cause: The new Cardinality Estimator (CE) model introduced in SQL Server 2014 (CE70 → CE120 → CE150 → CE160) can produce better or worse estimates depending on data distribution. Regression is common when moving across CE generations.

Strategy: Use Query Store to identify and fix regressions without rollback.

```
-- Enable Query Store if not already enabled (SQL Server 2016+)
```

```
ALTER DATABASE [YourDatabase] SET QUERY_STORE = ON (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    INTERVAL_LENGTH_MINUTES = 60,
    MAX_STORAGE_SIZE_MB = 1000,
    QUERY_CAPTURE_MODE = AUTO,     -- AUTO recommended in 2019+
    SIZE_BASED_CLEANUP_MODE = AUTO
);

-- Find regressed queries (QS Regressed Queries report equivalent in T-SQL)
SELECT TOP 20
    qt.query_sql_text,
    q.query_id,
    p.plan_id,
    rs_recent.avg_duration / 1000.0     AS avg_duration_ms_recent,
    rs_old.avg_duration / 1000.0        AS avg_duration_ms_baseline,
    rs_recent.avg_duration / NULLIF(rs_old.avg_duration, 0) AS regression_factor
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt  ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p         ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs_recent
    ON p.plan_id = rs_recent.plan_id
    AND rs_recent.last_execution_time > DATEADD(HOUR, -2, GETUTCDATE())
JOIN sys.query_store_runtime_stats rs_old
    ON p.plan_id = rs_old.plan_id
    AND rs_old.last_execution_time BETWEEN DATEADD(DAY, -7, GETUTCDATE())
                                       AND DATEADD(DAY, -1, GETUTCDATE())
WHERE rs_recent.avg_duration > 2 * rs_old.avg_duration  -- Regressed by 2x
ORDER BY regression_factor DESC;

-- Force a previously good plan for a regressed query
EXEC sys.sp_query_store_force_plan
    @query_id = 42,
    @plan_id = 7;   -- The plan_id that was fast before the upgrade
```

Graduated compatibility level migration (recommended approach):

1. Upgrade SQL Server version but keep database at old compat level temporarily.
2. Enable Query Store to capture baseline.
3. Use `ALTER DATABASE db SET COMPATIBILITY_LEVEL = <new>` on a non-production copy first.
4. Run representative workload; compare Query Store reports.

5. For regressions: use Query Store plan forcing or `USE HINT ('FORCE_LEGACY_CARDINALITY_ESTIMATION')` per query.
6. In SQL Server 2022+: CE Feedback automatically corrects poor CE assumptions per query at compat level 160.

Scenario 9 — Worker Thread Exhaustion (THREADPOOL Waits)

Real-world case: *Application reports connection timeouts. SQL Server is not under CPU or I/O pressure.* `THREADPOOL` *is the top wait. Max worker threads is at its limit.*

```
-- Check worker thread utilization
SELECT
    scheduler_count,
    max_workers_count,
    active_workers_count,
    (SELECT COUNT(*) FROM sys.dm_os_tasks WHERE task_state = 'PENDING') AS pendin
FROM sys.dm_os_sys_info;

-- Look at scheduler pressure
SELECT
    scheduler_id,
    cpu_id,
    status,
    runnable_tasks_count,
    work_queue_count,
    pending_disk_io_count
FROM sys.dm_os_schedulers
WHERE status = 'VISIBLE ONLINE'
ORDER BY runnable_tasks_count DESC;
```

Root causes:

- Too many short-lived connections (connection pool not configured correctly on application side)
- Parallel queries consuming too many workers per request (each parallel thread = one worker)
- Blocking chain where many sessions queue behind a single head blocker, each holding a worker

Fix sequence:

1. Address the blocking chain first (usually the immediate cause).

2. Increase MAXDOP scoped configuration to reduce workers per query.
3. Configure application connection pool with appropriate min/max settings.
4. On SQL Server 2019+, consider `MAX_DISPATCH_LATENCY` and DOP feedback.

```
-- Raise max worker threads only as last resort and after root cause is addressed
-- Default (0) = SQL Server auto-configures; formula: 512 + (CPUs - 4) * 16
EXEC sp_configure 'max worker threads', 0;  -- Reset to auto
RECONFIGURE;
```

---

6. Phase 5 — Intelligent Query Processing (SQL Server 2019–2025)

These features work automatically when the database compatibility level is set correctly. Understanding them prevents misdiagnosis (“the engine is still slow”) and unnecessary manual tuning.

Feature Matrix by Version

Feature	SQL Server 2019 (150)	SQL Server 2022 (160)	SQL Server 2025
Adaptive Joins	✅	✅	✅
Memory Grant Feedback	✅ Row + Batch	✅ Percentile feedback	✅ Enhanced
Batch Mode on Rowstore	✅	✅	✅
Table Variable Deferred Compilation	✅	✅	✅
Scalar UDF Inlining	✅	✅	✅
Parameter Sensitive Plan (PSP) Optimization	❌	✅	✅
Cardinality Estimation (CE) Feedback	❌	✅	✅ Enhanced (expressions)
DOP Feedback	❌	✅ (Always-up-to-date policy)	✅

Query Store for Secondary Replicas	❌	✅	✅
Optimized Plan Forcing	❌	✅	✅
AI-driven Automatic Tuning	❌	❌	✅
TempDB Memory-Optimized Metadata	✅	✅	✅

Enabling IQP Features

```
-- Verify database compatibility level
SELECT name, compatibility_level FROM sys.databases WHERE name = DB_NAME();

-- SQL Server 2022 - set to level 160 to enable all IQP features
ALTER DATABASE [YourDatabase] SET COMPATIBILITY_LEVEL = 160;

-- Check which IQP features are enabled/disabled for a database
SELECT * FROM sys.database_scoped_configurations
WHERE name IN (
    'PARAMETER_SENSITIVE_PLAN_OPTIMIZATION',
    'OPTIMIZED_PLAN_FORCING',
    'DOP_FEEDBACK',
    'CE_FEEDBACK'
);
```

Memory Grant Feedback in Practice

SQL Server 2019: Grant feedback adjusts memory per query between executions to stop spills. **SQL Server 2022:** Percentile-based grant feedback uses a smarter rolling model so it doesn't overcorrect.

```
-- See which queries are benefiting from Memory Grant Feedback
SELECT
    qt.query_sql_text,
    q.query_id,
    p.plan_id,
    p.is_feedback_adjusted,        -- 1 = plan was adjusted by feedback
    p.is_memory_grant_feedback_adjusted
FROM sys.query_store_plan p
JOIN sys.query_store_query q   ON p.query_id = q.query_id
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
WHERE p.is_feedback_adjusted = 1;
```

PSP Optimization (SQL Server 2022+)

PSP creates multiple plan variants ("dispatcher" plans) for a parameterized query when SQL Server detects data skew. A single stored procedure can now serve a range of parameter inputs with appropriate plans — without the DBA manually adding `OPTION(RECOMPILE)` or query hints.

```
-- Verify PSP is in use for a query (look for dispatcher plan)
SELECT
    qt.query_sql_text,
    p.plan_id,
    p.query_plan_hash,
    p.is_parameterizable,
    p.plan_forcing_type_desc
FROM sys.query_store_plan p
JOIN sys.query_store_query q   ON p.query_id = q.query_id
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
WHERE qt.query_sql_text LIKE '%usp_YourProcedure%';
-- Multiple plan_ids for the same query_id indicate PSP variants are active
```

---

7. Phase 6 — Targeted Remediation and Validation

The Change Protocol

Every change in production must follow this sequence:

```
1. State the hypothesis clearly.
   "I believe CPU spikes are caused by procedure X using a parallel plan
    with MAXDOP=0. Reducing MAXDOP to 4 at the database scope should
    reduce CPU by ~40%."

2. Identify the validation metric.
   "After the change, avg CPU should drop from 85% to <50% during peak hours.
    Execution time for X should remain < 2 seconds."

3. Make one change at a time.
   Never apply multiple changes simultaneously. You won't know which one worked.

4. Capture before and after evidence.
   Snapshot sys.dm_os_wait_stats, Query Store metrics, or PerfMon counters
   before and 30 minutes after.
```

```
5. Document the change with timestamp, reason, and expected outcome.
   "14:22 UTC — Set MAXDOP = 4 on ORDERS database.
    Expected: reduce SOS_SCHEDULER_YIELD waits by 40%."

6. Define rollback criteria.
   "If avg CPU doesn't drop within 20 minutes, or any other metric degrades
    by >20%, revert immediately."
```

Safe Quick-Win Remediations

Remediation	Risk	Reversible?	Notes
Update statistics with FULLSCAN	Low	N/A (no rollback needed)	Always safe; can improve plan quality immediately
Enable RCSI (Read Committed Snapshot Isolation)	Medium	Yes	Requires temp lock; uses version store in TempDB
Force a plan via Query Store	Low	Yes — one click	Best first option for regression
Add TempDB files	Low	Yes (remove file, but requires shrink)	Do not auto-shrink TempDB afterward
Add a nonclustered index	Medium	Yes (drop index)	Test write overhead; don't add speculatively
Raise cost threshold for parallelism	Low	Yes (sp_configure)	25–50 is the typical safe range
Enable Memory-Optimized TempDB metadata	Medium	Yes (requires restart)	SQL Server 2019+; high-concurrency OLTP only

Validation Queries (Before / After Comparison)

```
-- Snapshot wait stats to a temp table (before change)
SELECT GETUTCDATE() AS snapshot_time, *
INTO #waits_before
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN ('SLEEP_TASK','WAITFOR','XE_TIMER_EVENT');

-- (Make the change)
```

```
-- Compare after 10-30 minutes
SELECT
    a.wait_type,
    (a.wait_time_ms - b.wait_time_ms) / 1000.0      AS delta_wait_sec,
    (a.waiting_tasks_count - b.waiting_tasks_count)  AS delta_tasks
FROM sys.dm_os_wait_stats a
JOIN #waits_before b ON a.wait_type = b.wait_type
WHERE (a.wait_time_ms - b.wait_time_ms) > 1000
ORDER BY delta_wait_sec DESC;
```

---

8. Extended Events: Real-Time Capture Without Profiler

SQL Profiler is deprecated. Extended Events (XE) is the correct tool from SQL Server 2012 onwards. In SQL Server 2025, the new `MAX_DURATION` option for XE sessions stops sessions automatically after a time limit — critical for production.

Capture Slow Queries in Real Time

```
-- Capture queries running longer than 5 seconds
-- SQL Server 2016+ syntax
CREATE EVENT SESSION [SlowQueries] ON SERVER
ADD EVENT sqlserver.sql_statement_completed (
    ACTION (
        sqlserver.sql_text,
        sqlserver.plan_handle,
        sqlserver.session_id,
        sqlserver.database_name,
        sqlserver.username
    )
    WHERE duration > 5000000  -- microseconds; 5,000,000 = 5 seconds
),
ADD EVENT sqlserver.rpc_completed (
    ACTION (sqlserver.sql_text, sqlserver.plan_handle, sqlserver.session_id)
    WHERE duration > 5000000
)
ADD TARGET package0.ring_buffer (SET max_memory = 102400)  -- 100 MB ring buffer
-- SQL Server 2025+: add MAX_DURATION to auto-stop the session
-- WITH (MAX_DISPATCH_LATENCY = 5 SECONDS, STARTUP_STATE = OFF, MAX_DURATION = 60
WITH (MAX_DISPATCH_LATENCY = 5 SECONDS, STARTUP_STATE = OFF);

ALTER EVENT SESSION [SlowQueries] ON SERVER STATE = START;
```

Read the Ring Buffer Results

```
SELECT
    event_data.value('(@timestamp)[1]', 'datetime2')          AS event_time,
    event_data.value('(data[@name="duration"]/value)[1]', 'bigint') / 1000000.0 A
    event_data.value('(action[@name="sql_text"]/value)[1]', 'nvarchar(max)') AS s
    event_data.value('(action[@name="database_name"]/value)[1]', 'nvarchar(128)')
FROM (
    SELECT CAST(target_data AS XML) AS ring_buffer
    FROM sys.dm_xe_session_targets t
    JOIN sys.dm_xe_sessions s ON t.event_session_address = s.address
    WHERE s.name = 'SlowQueries' AND t.target_name = 'ring_buffer'
) rb
CROSS APPLY ring_buffer.nodes('//RingBufferTarget/event') AS events(event_data)
ORDER BY duration_sec DESC;

-- Clean up
ALTER EVENT SESSION [SlowQueries] ON SERVER STATE = STOP;
DROP EVENT SESSION [SlowQueries] ON SERVER;
```

---

9. Query Store: Your Built-in Performance History

Query Store is the most important DBA tool introduced since Extended Events. Enabled by default in SQL Server 2016+ (required-on in Azure SQL Database). Think of it as a “flight recorder” for query performance.

Essential Query Store Queries

```
-- Top 10 queries by total CPU in the last 4 hours
SELECT TOP 10
    qt.query_sql_text,
    q.query_id,
    SUM(rs.count_executions)                    AS total_executions,
    AVG(rs.avg_cpu_time) / 1000.0               AS avg_cpu_ms,
    SUM(rs.avg_cpu_time * rs.count_executions) / 1000000.0 AS total_cpu_sec,
    AVG(rs.avg_duration) / 1000.0               AS avg_duration_ms
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q   ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p    ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs   ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi
    ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
WHERE rsi.start_time > DATEADD(HOUR, -4, GETUTCDATE())
```

```
GROUP BY qt.query_sql_text, q.query_id
ORDER BY total_cpu_sec DESC;

-- Queries with multiple plans (plan instability — candidate for plan forcing)
SELECT
    q.query_id,
    qt.query_sql_text,
    COUNT(DISTINCT p.plan_id)   AS plan_count,
    MIN(rs.avg_duration) / 1000.0 AS best_avg_ms,
    MAX(rs.avg_duration) / 1000.0 AS worst_avg_ms
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p        ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
GROUP BY q.query_id, qt.query_sql_text
HAVING COUNT(DISTINCT p.plan_id) > 1
   AND MAX(rs.avg_duration) > 3 * MIN(rs.avg_duration)
ORDER BY worst_avg_ms DESC;

-- Unforce a plan if behavior improves (remove the forced plan)
EXEC sys.sp_query_store_unforce_plan @query_id = 42, @plan_id = 7;

-- Purge stale data (run during maintenance)
EXEC sys.sp_query_store_flush_db;
```

Query Store on Secondary Replicas (SQL Server 2022+)

```
-- Enable QS for readable secondaries (AG environments)
ALTER DATABASE [YourDatabase]
SET QUERY_STORE = ON (QUERY_CAPTURE_MODE = AUTO, READ_WRITE_MODE = READ_WRITE);

-- On 2022+: data from secondaries is aggregated into primary's Query Store
-- Check which replica data came from
SELECT
    p.plan_id,
    rs.replica_group_id,   -- NULL = primary; non-NULL = secondary replica
    rs.avg_duration / 1000.0 AS avg_duration_ms
FROM sys.query_store_runtime_stats rs
JOIN sys.query_store_plan p ON rs.plan_id = p.plan_id;
```

---

10. Version-by-Version Feature Reference (2016–2025)

Capability	2016 (130)	2017 (140)	2019 (150)	2022 (160)	2025
Query Store	✅ Basic	✅ + Spill stats	✅ + AUTO mode	✅ + Secondary replicas	✅ + AI recommendations
Live Query Statistics	✅	✅	✅	✅	✅
Automatic Plan Correction	❌	✅	✅	✅	✅ Enhanced
Memory Grant Feedback	❌	❌	✅ Batch+Row	✅ Percentile	✅
Adaptive Joins	❌	✅	✅	✅	✅
PSP Optimization	❌	❌	❌	✅	✅
CE Feedback	❌	❌	❌	✅	✅ + Expressions
DOP Feedback	❌	❌	❌	✅	✅
Memory-Opt TempDB Metadata	❌	❌	✅	✅	✅
XE MAX_DURATION	❌	❌	❌	❌	✅
RCSI	✅	✅	✅	✅	✅
Resumable Index Operations	❌	✅ Online	✅ Offline too	✅	✅
Accelerated Database Recovery (ADR)	❌	❌	✅	✅	✅
Graph Queries	❌	✅	✅	✅	✅
Columnstore IQP	❌	❌	✅ Batch mode	✅	✅

11. Escalation Decision Tree

```
START: Performance complaint received
         |
         ▼
Is the instance completely unresponsive?
├── YES → Check THREADPOOL waits, check blocking chain depth,
|         consider DAC connection (port 1434), check SQL Server error log
└── NO
         |
         ▼
Is the problem new (< 24 hours)?
├── YES → What changed? (deployment, job, schema, stats, data volume)
|         → Start with Query Store regression report
└── NO (chronic/gradual)
         → Compare to baseline; check data volume growth, index fragmentation
         |
         ▼
Is only ONE query/procedure affected?
├── YES → Get execution plan; check for:
|         - Parameter sniffing (execution plan for wrong parameter set)
|         - Outdated statistics (stale rowcount estimates)
|         - Index scan instead of seek (missing index or leading column mismatch)
└── NO (multiple queries or whole system)
         |
         ▼
Check sys.dm_os_wait_stats dominant wait type:
├── SOS_SCHEDULER_YIELD / CXPACKET → CPU / parallelism → check MAXDOP, top CPU qu
├── PAGEIOLATCH_* / WRITELOG → Storage I/O → check dm_io_virtual_file_stats
├── RESOURCE_SEMAPHORE → Memory grants → check max server memory, buffer pool
├── LCK_M_* → Blocking → find head blocker, check transaction age
├── THREADPOOL → Worker exhaustion → check max workers, blocking chains, MAXDOP
└── ASYNC_NETWORK_IO → Application-layer bottleneck → not a SQL Server issue
         |
         ▼
Make one targeted, documented change → Validate → Repeat if needed
```

---

12. Quick-Reference Checklist

Immediate Response (First 5 Minutes)

- State the problem with time window, affected objects, and change context

- Run `sys.dm_exec_requests` — identify active sessions with high wait_time
- Check `blocking_session_id` — is there a blocking chain?
- Check top wait types via `sys.dm_os_wait_stats`
- Check signal wait percentage (CPU pressure indicator)

Investigation (5–30 Minutes)

- Narrow scope: instance-wide vs. single database vs. single query
- Establish baseline: is this actually abnormal for this system/time?
- Check Query Store regressed queries report
- Run missing index DMV for the affected database
- Pull execution plan for the specific query (actual, not estimated)
- Check statistics last updated date on involved tables
- Check `sys.dm_io_virtual_file_stats` for storage latency > 15ms

Before Any Change

- Document hypothesis and expected outcome
- Snapshot wait stats for before/after comparison
- Identify rollback procedure
- Make one change at a time

Post-Change Validation

- Compare wait stats delta (before/after snapshot)
- Verify query duration in Query Store runtime stats
- Confirm the original symptom is resolved (not just masked)
- Document outcome in incident ticket

References

- sys.dm_os_wait_stats — Microsoft Learn

- Intelligent Query Processing — Microsoft Learn
- Monitoring Performance with Query Store — Microsoft Learn
- Troubleshoot Slow SQL Server I/O — Microsoft Learn
- sys.dm_exec_requests — Microsoft Learn
- SQL Server 2025 Query Performance Tuning — Grant Fritchey, Apress 2026
- CraftedSQL: Bottleneck Analysis — Björn Peters

SOP Version: 2.0 | Covers SQL Server 2016–2025 | Last reviewed: May 2026 Internal review cycle: Quarterly or after any SQL Server major version upgrade